

TABLE OF CONTENTS

TABLE OF CONTENTS

	1
2) a C Source Code	2
b) Terminal Execution & Verification	2
3) a C Source Code	3
b) Terminal Execution & Verification	3
4) a C Source Code	4
b) Terminal Execution & Verification	4
5) a C Source Code	4
6) a C Source Code	5
b) Terminal Execution & Verification	5

LIST OF FIGURES

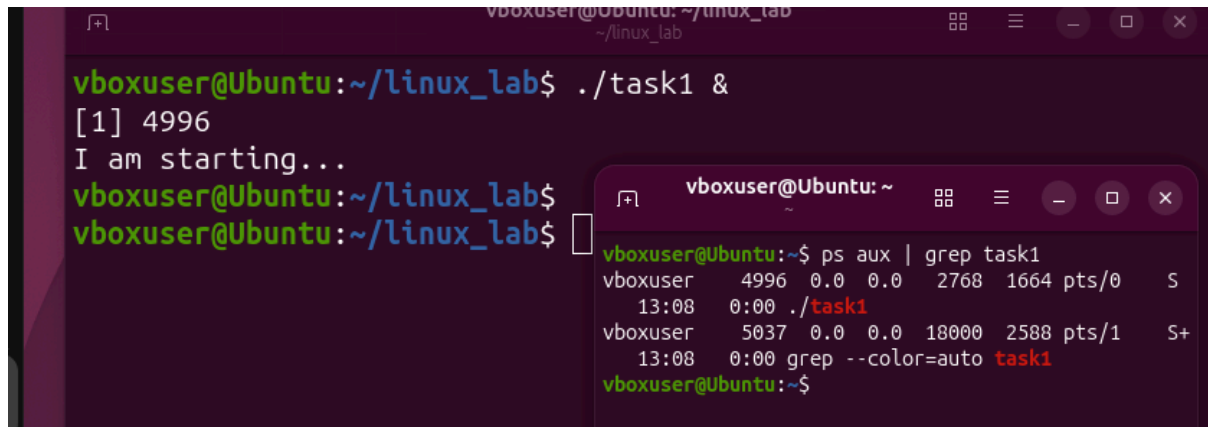
- 1) Figure 1: C Source Code Compilation and Execution
- 2) Figure 2: Terminal Execution and Process Verification Using ps Command
- 3) Figure 3: C Source Code and Program Execution
- 4) Figure 4: Terminal Execution and PID/PPID Verification
- 5) Figure 5: C Program Execution and Exit Status Verification
- 6) Figure 6: Failure Case and Exit Status Verification
- 7) Figure 7: C Source Code and Program Execution (Task 4)
- 8) Figure 8: C Source Code and Initial Program Execution
- 9) Figure 9: Terminal Execution and Process Continuation/Termination Verification

2) a C Source Code

```
vboxuser@Ubuntu:~/linux_lab$ nano task1.c
vboxuser@Ubuntu:~/linux_lab$ gcc task1.c -o task1
vboxuser@Ubuntu:~/linux_lab$ ./task1
I am starting...
I am finished.
vboxuser@Ubuntu:~/linux_lab$
```

Figure 1: C Source Code Compilation and Execution

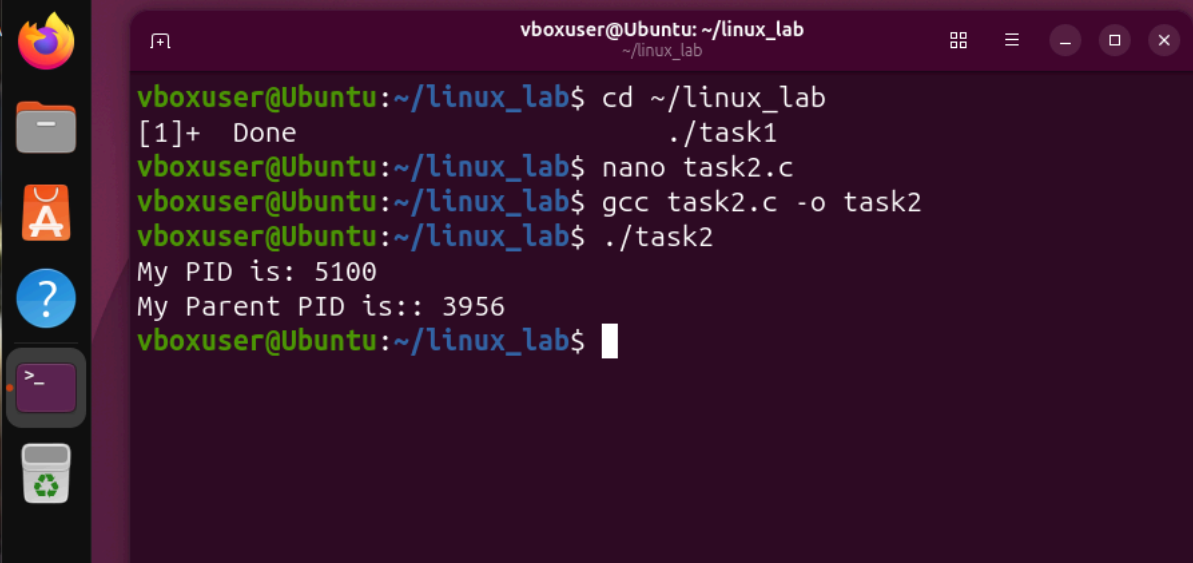
b) Terminal Execution & Verification

The image shows a terminal window with a dark purple background. The user 'vboxuser' is in the directory '~/linux_lab'. They run './task1 &', which returns '[1] 4996' and prints 'I am starting...'. Then they run 'ps aux | grep task1', which shows two processes: one for './task1' (PID 4996) and one for 'grep' (PID 5037).

```
vboxuser@Ubuntu:~/linux_lab$ ./task1 &
[1] 4996
I am starting...
vboxuser@Ubuntu:~/linux_lab$ ps aux | grep task1
vboxuser  4996  0.0  0.0  2768 1664 pts/0    S+   13:08   0:00  ./task1
vboxuser  5037  0.0  0.0  18000 2588 pts/1    S+   13:08   0:00  grep --color=auto task1
vboxuser@Ubuntu:~$
```

Figure 2: Terminal Execution and Process Verification Using ps Command

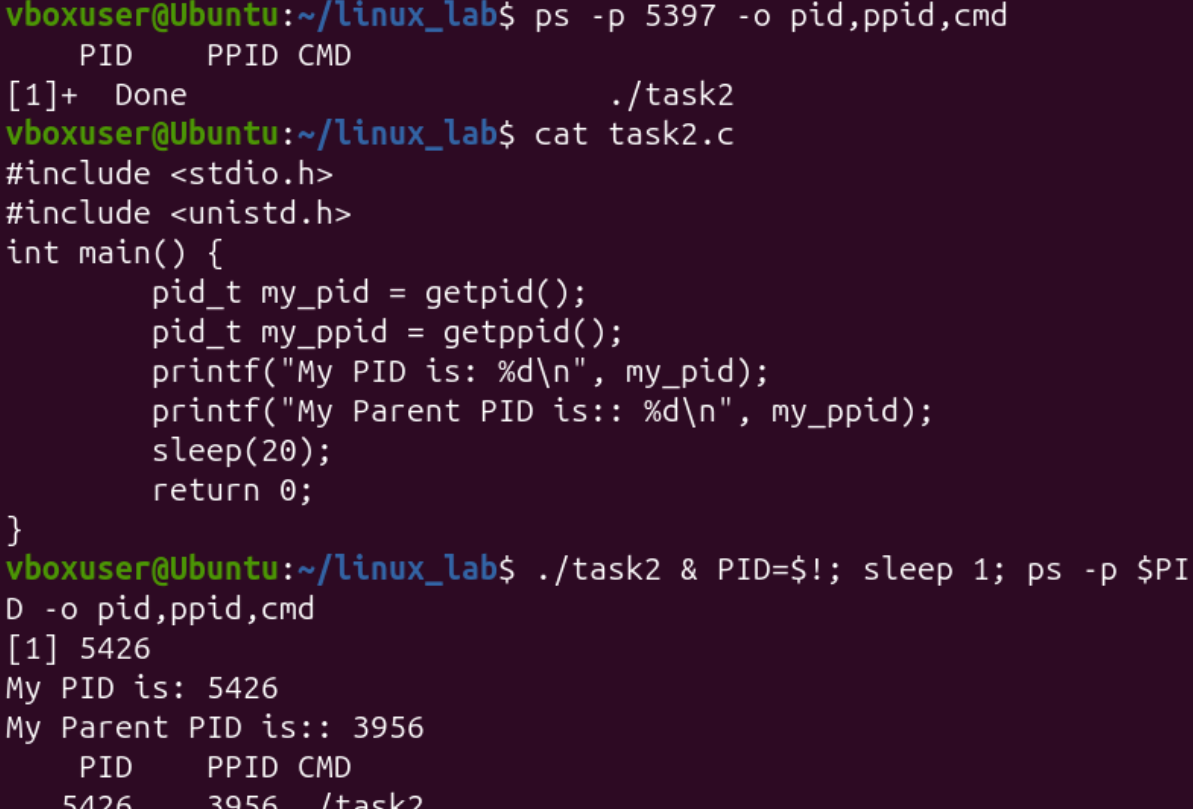
3) a C Source Code

A terminal window titled 'vboxuser@Ubuntu: ~/linux_lab' with standard window controls. The terminal shows a sequence of commands: 'cd ~/linux_lab', './task1', 'nano task2.c', 'gcc task2.c -o task2', and './task2'. The output of './task2' displays 'My PID is: 5100' and 'My Parent PID is:: 3956'.

```
vboxuser@Ubuntu:~/linux_lab$ cd ~/linux_lab
[1]+  Done                  ./task1
vboxuser@Ubuntu:~/linux_lab$ nano task2.c
vboxuser@Ubuntu:~/linux_lab$ gcc task2.c -o task2
vboxuser@Ubuntu:~/linux_lab$ ./task2
My PID is: 5100
My Parent PID is:: 3956
vboxuser@Ubuntu:~/linux_lab$
```

Figure 3: C Source Code and Program Execution

b) Terminal Execution & Verification

A terminal window showing the verification of PID and PPID. It starts with 'ps -p 5397 -o pid,ppid,cmd' showing a process with PID 5397. Then, 'cat task2.c' displays the source code of task2.c, which prints its own PID and PPID and sleeps for 20 seconds. Finally, './task2 & PID=\$!; sleep 1; ps -p \$PID -o pid,ppid,cmd' is executed, resulting in a new process with PID 5426 and PPID 3956.

```
vboxuser@Ubuntu:~/linux_lab$ ps -p 5397 -o pid,ppid,cmd
  PID   PPID  CMD
[1]+  Done                  ./task2
vboxuser@Ubuntu:~/linux_lab$ cat task2.c
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();
    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is:: %d\n", my_ppid);
    sleep(20);
    return 0;
}
vboxuser@Ubuntu:~/linux_lab$ ./task2 & PID=$!; sleep 1; ps -p $PID -o pid,ppid,cmd
[1] 5426
My PID is: 5426
My Parent PID is:: 3956
  PID   PPID  CMD
 5426   3956  ./task2
```

Figure 4: Terminal Execution and PID/PPID Verification

4) a C Source Code

```
vboxuser@Ubuntu:~/linux_lab$ nano task3.c
vboxuser@Ubuntu:~/linux_lab$ gcc task3.c -o task3
vboxuser@Ubuntu:~/linux_lab$ ./task3
Enter a number (positive for success, negative for fail); 5
Success!
vboxuser@Ubuntu:~/linux_lab$ echo $?
0
vboxuser@Ubuntu:~/linux_lab$
```

Figure 5: C Program Execution and Exit Status Verification

b) Terminal Execution & Verification

```
vboxuser@Ubuntu:~/linux_lab$ ./task3
Enter a number (positive for success, negative for fail); -5
Failure!
vboxuser@Ubuntu:~/linux_lab$ echo $?
1
vboxuser@Ubuntu:~/linux_lab$
```

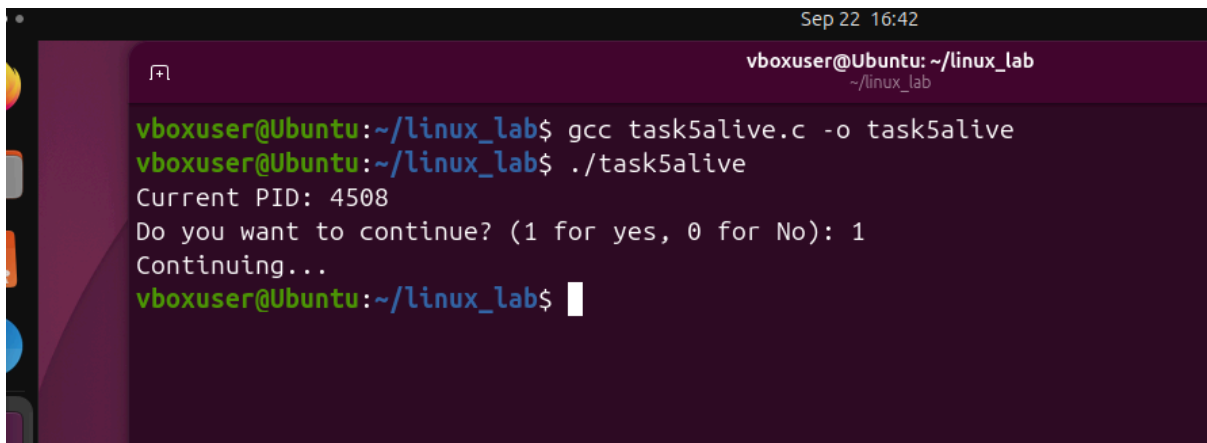
Figure 6: Failure Case and Exit Status Verification

5) a C Source Code

```
vboxuser@Ubuntu: ~/linux_lab
vboxuser@Ubuntu:~/linux_lab$ nano task4.c
vboxuser@Ubuntu:~/linux_lab$ gcc task4.c -o task4
vboxuser@Ubuntu:~/linux_lab$ ./task4
Enter your name: Prathana
Hello, Prathana! Welcome to OS Class.
vboxuser@Ubuntu:~/linux_lab$
```

Figure 7: C Source Code and Program Execution (Task 4)

6) a C Source Code

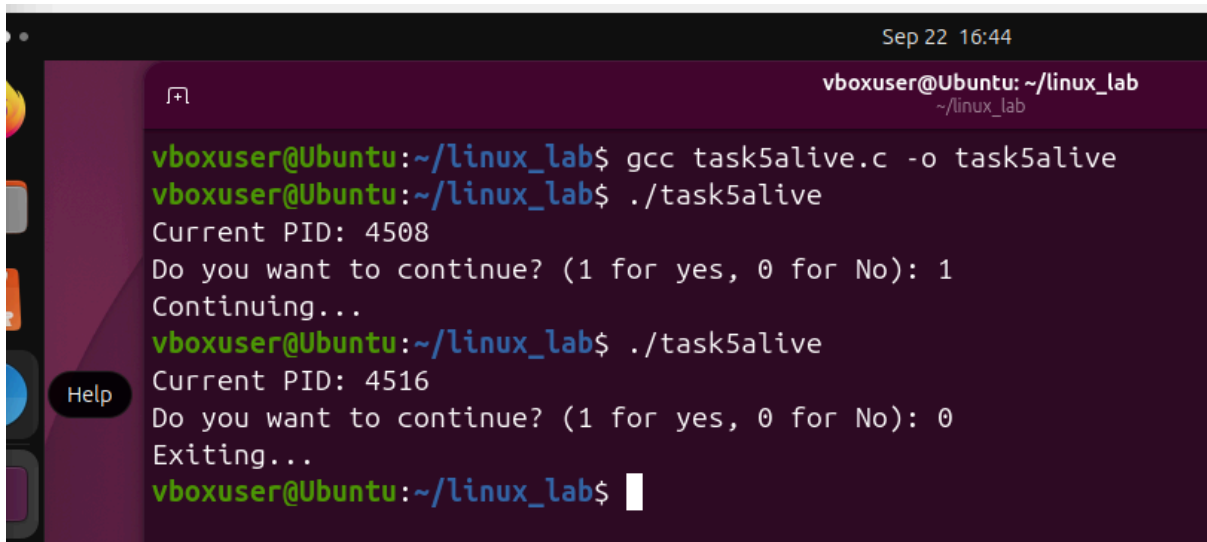


```
Sep 22 16:42
vboxuser@Ubuntu: ~/linux_lab

vboxuser@Ubuntu:~/linux_lab$ gcc task5alive.c -o task5alive
vboxuser@Ubuntu:~/linux_lab$ ./task5alive
Current PID: 4508
Do you want to continue? (1 for yes, 0 for No): 1
Continuing...
vboxuser@Ubuntu:~/linux_lab$
```

Figure 8: C Source Code and Initial Program Execution

b) Terminal Execution & Verification



```
Sep 22 16:44
vboxuser@Ubuntu: ~/linux_lab

vboxuser@Ubuntu:~/linux_lab$ gcc task5alive.c -o task5alive
vboxuser@Ubuntu:~/linux_lab$ ./task5alive
Current PID: 4508
Do you want to continue? (1 for yes, 0 for No): 1
Continuing...
vboxuser@Ubuntu:~/linux_lab$ ./task5alive
Current PID: 4516
Do you want to continue? (1 for yes, 0 for No): 0
Exiting...
vboxuser@Ubuntu:~/linux_lab$
```

Figure 9: Terminal Execution and Process Continuation/Termination Verification